logistiklabb

November 6, 2026

1 Logistiklabb 2026

Grupp: Rikard & David

[88]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
pd.set_option("display.precision", 2)
```

---

2 1. Lagerstyrning och produktion

2.1 1a) Exponentiell utjämning

Prognosen uppdateras med

$$u(t) = a\,d(t) + (1-a)\,u(t-1), \qquad 0 \le a \le 1,$$

där $u(t-1)$ är prognosen för månad t (gjord månaden innan) och $u(0) = 50$ är startskattningen (prognos för januari). Prognosen för december är alltså $u(\text{nov})$.

Prognosfelet mäts med MAPE över de 11 månader där vi har utfall:

$$\text{MAPE} = \frac{100}{n}\sum_{t=1}^{n}\frac{|d(t) - u(t-1)|}{d(t)}\ \%.$$

[89]:
```
manader = ␣
 ↪["jan","feb","mar","apr","maj","jun","jul","aug","sep","okt","nov","dec"]
d = np.array([52, 51, 60, 46, 70, 50, 39, 48, 60, 70, 60], dtype=float)
u0 = 50.0

def utjamning(a, d=d, u0=u0):
    # Returnerar prognoserna u(t-1) för jan..dec (12 värden)
    u = [u0]
    for dt in d:
        u.append(a*dt + (1-a)*u[-1])
    return np.array(u)
```

```
def mape(a):
    u = utjamning(a)
    return 100*np.mean(np.abs(d - u[:-1]) / d)

alfa = np.round(np.arange(0, 1.01, 0.1), 1)
tabell = pd.DataFrame({
    "a": alfa,
    "Prognos dec": [utjamning(a)[-1] for a in alfa],
    "MAPE (%)": [mape(a) for a in alfa],
})
tabell
```

[89]:

	a	Prognos dec	MAPE (%)
0	0.0	50.00	14.00
1	0.1	54.15	14.45
2	0.2	56.66	14.93
3	0.3	58.48	15.26
4	0.4	59.90	15.37
5	0.5	60.94	16.20
6	0.6	61.56	17.31
7	0.7	61.74	18.29
8	0.8	61.49	19.11
9	0.9	60.89	19.79
10	1.0	60.00	20.31

[90]:
```
# Prognoserna månad för månad för några värden på a
prognoser = pd.DataFrame({"d(t)": list(d) + [np.nan]}, index=manader)
for a in [0.1, 0.3, 0.5, 0.9]:
    prognoser[f"u(t-1), a={a}"] = utjamning(a)
prognoser.T
```

[90]:

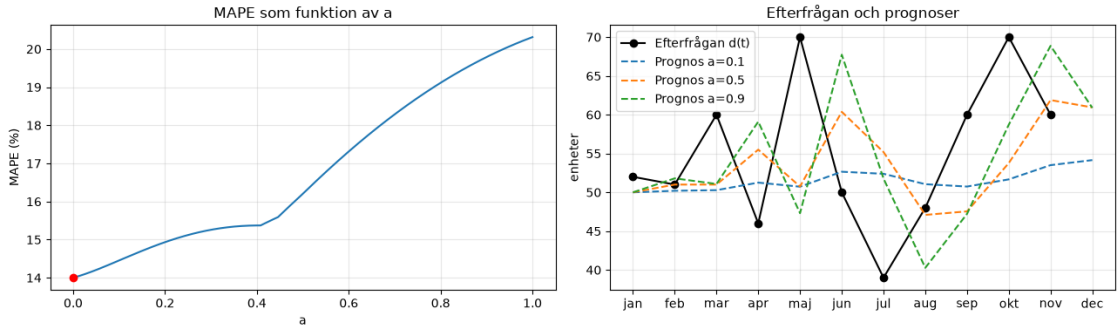
	jan	feb	mar	apr	maj	jun	jul	aug	sep	okt	nov	dec
d(t)	52.0	51.0	60.00	46.00	70.00	50.00	39.00	48.00	60.00	70.00	60.00	NaN
u(t-1), a=0.1	50.0	50.2	50.28	51.25	50.73	52.65	52.39	51.05	50.74	51.67	53.50	54.15
u(t-1), a=0.3	50.0	50.6	50.72	53.50	51.25	56.88	54.81	50.07	49.45	52.61	57.83	58.48
u(t-1), a=0.5	50.0	51.0	51.00	55.50	50.75	60.38	55.19	47.09	47.55	53.77	61.89	60.94
u(t-1), a=0.9	50.0	51.8	51.08	59.11	47.31	67.73	51.77	40.28	47.23	58.72	68.87	60.89

[91]:
```
fin = np.linspace(0, 1, 1001)
m = np.array([mape(a) for a in fin])
a_best = fin[m.argmin()]
print(f"Lägst MAPE = {m.min():.2f} % vid a = {a_best:.3f}, prognos dec =␣
 ↪{utjamning(a_best)[-1]:.2f}")

fig, ax = plt.subplots(1, 2, figsize=(13, 4))
ax[0].plot(fin, m)
ax[0].scatter([a_best], [m.min()], color="red", zorder=3)
ax[0].set(xlabel="a", ylabel="MAPE (%)", title="MAPE som funktion av a")
ax[0].grid(alpha=.3)

ax[1].plot(manader[:11], d, "ko-", label="Efterfrågan d(t)")
for a in [0.1, 0.5, 0.9]:
    ax[1].plot(manader, utjamning(a), "--", label=f"Prognos a={a}")
ax[1].set(title="Efterfrågan och prognoser", ylabel="enheter")
ax[1].legend(); ax[1].grid(alpha=.3)
plt.tight_layout(); plt.show()
```

```
Lägst MAPE = 14.00 % vid a = 0.000, prognos dec = 50.00
```



Analys (att fylla i / diskutera): - *Litet* $a \to$ prognosen lägger lite vikt vid senaste utfallet och mycket vid historiken: jämn, trög prognos som filtrerar bort slumpvariation. $a = 0$ ger alltid 50. - *Stort* $a \to$ prognosen följer senaste utfallet nära: snabb reaktion på förändringar men också på brus. $a = 1$ ger "naiv prognos" $u(t) = d(t)$. - Efterfrågan här svänger kraftigt utan tydlig trend, så … *(kommentera vilket a som gav lägst MAPE och varför).*

2.1.1 Härledning av den rekursiva formeln

Utgå från den viktade summan av hela tidsserien där vikterna avtar geometriskt:

$$u(t) = a\,d(t) + a(1-a)\,d(t-1) + a(1-a)^2 d(t-2) + \cdots + (1-a)^t u(0) = a\sum_{k=0}^{t-1}(1-a)^k d(t-k) + (1-a)^t u(0).$$

Skriv motsvarande uttryck för $u(t-1)$ och multiplicera med $(1-a)$:

$$(1-a)\,u(t-1) = a\sum_{k=0}^{t-2}(1-a)^{k+1}d(t-1-k) + (1-a)^t u(0) = a\sum_{k=1}^{t-1}(1-a)^k d(t-k) + (1-a)^t u(0).$$

Det är exakt alla termer i $u(t)$ utom den första, $a\,d(t)$. Alltså

$$u(t) = a\,d(t) + (1-a)\,u(t-1).$$

Vikterna $a(1-a)^k$ summerar (tillsammans med $(1-a)^t$) till 1, så det är ett riktigt viktat medelvärde där äldre observationer väger exponentiellt mindre.

2.2 1b) ABC-analys

Lagervärde per artikel = antal × medelkostnad. Artiklarna sorteras efter lagervärde och den kumulativa andelen beräknas för att göra analysen. Gränsen dras vid A = de första 80 % av värdet, B = 80-95 % och C de sista 5 %.

2.2.1 Lösning i Excel

Analysen är gjord i Excel och arbetsgången är följande:

1. Lagervärdet räknas ut för varje artikel och tabellen sorteras fallande efter lagervärde.
2. Lagervärdet summeras kumulativt nedåt i tabellen.
3. Den kumulativa summan delas med det totala lagervärdet, vilket ger kumulativ andel.

Kolumn	Innehåll	Formel (rad 3 som exempel)
D	Lagervärde	`=B3*C3` (antal × medelkostnad)
E	Kumulativt lagervärde	`=E2+D3` (i E2: `=D2`)
F	Kumulativ andel	`=E3/E26` (E26 = totalt lagervärde)
G	ABC-klass	`=OM(F3<=0,8; "A"; OM(F3<=0,95; "B"; "C"))`

En vara klassas som A om kumulativa andelen är mindre än 80 %, B om den är mindre än 95% och annars som C.

Nedan läses Excels resultat in och formlerna hämtas direkt ur filen.

[92]:
```
import openpyxl

# Värdena som Excel har räknat fram
abc = pd.read_excel("abc.xlsx", sheet_name="Blad1", header=0, usecols="A:G",␣
 ↪nrows=25)
abc.columns = ["vara", "antal", "medelkostnad", "lagervärde", "kumulativt",␣
 ↪"kumulativ", "klass"]
abc.index = range(1, len(abc) + 1)

# Formlerna som står i Excel-arket (rad 3 som exempel)
ws = openpyxl.load_workbook("abc.xlsx")["Blad1"]
```

```
for kol in "DEFG":
    print(f"{kol}3: {ws[f'{kol}3'].value}")

abc.style.format({"lagervärde": "{:,.0f}", "kumulativt": "{:,.0f}", "kumulativ":␣
↪"{:.2%}"})
```

```
D3: =B3*C3
E3: =E2+D3
F3: =E3/$E$26
G3: =IF(F3<=0.8,"A",IF(F3<=0.95,"B","C"))
```

[92]: <pandas.io.formats.style.Styler at 0x1bf720307d0>

[93]:
```
sammanfattning = abc.groupby("klass").agg(
    antal_artiklar=("vara", "count"),
    lagervärde=("lagervärde", "sum"),
)
sammanfattning["andel artiklar (%)"] = 100 * sammanfattning["antal_artiklar"] /␣
↪len(abc)
sammanfattning["andel lagervärde (%)"] = 100 * sammanfattning["lagervärde"] /␣
↪abc["lagervärde"].sum()
sammanfattning
```

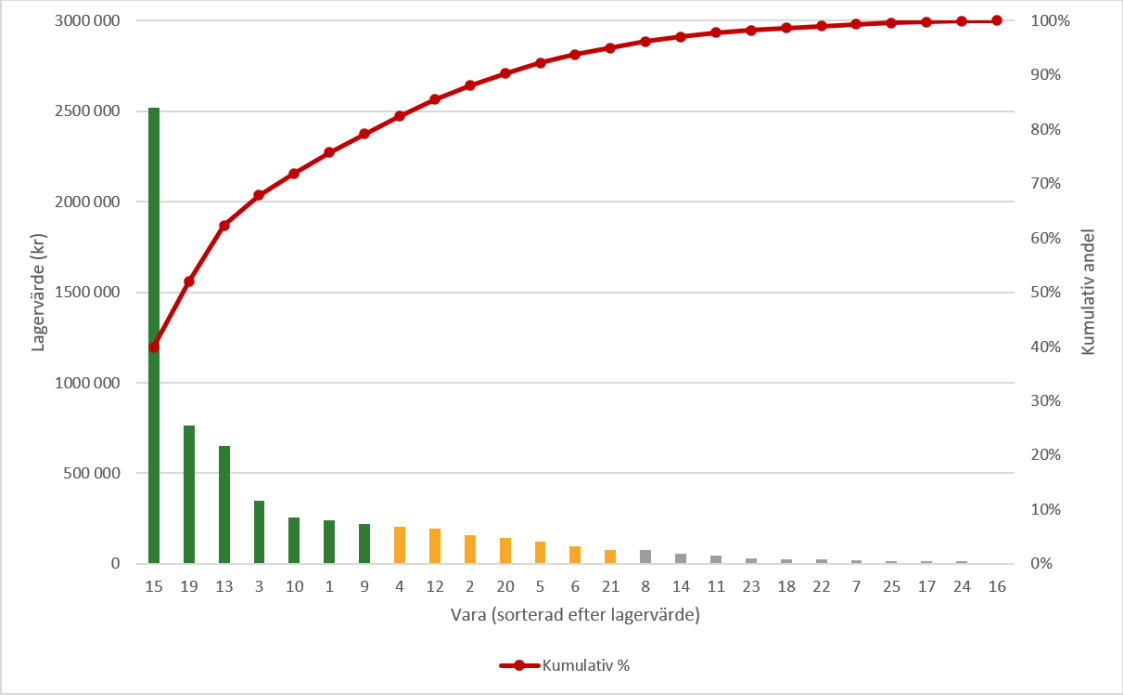
[93]:

klass	antal_artiklar	lagervärde	andel artiklar (%)	andel lagervärde (%)
A	7	4999559	28.0	79.18
B	7	996852	28.0	15.79
C	11	317420	44.0	5.03

Det blev 7 A-artiklar som står för ca 79% av hela lagervärdet. Vilket kommer att innebära tät uppföljning, noggranna prognoser och beställningar för just dessa varor. B och speciellt C-artiklar klarar sig med enkla regler, större partier och mindre kontroll.

[ ]:

Här ser vi det illustrerat hur bara 7 varor står för nästan 80 % av lagervärdet.



Figur 1. Paretodiagram över ABC-analysen, gjort i Excel.

---

3 2. Heltalsprogrammering (tilldelning av arbetsuppgifter)

3.0.1 2a) Modell och lösning

TODO: mängder, parametrar, binära variabler x_{ij}, målfunktion och bivillkor.

3.0.2 2b) Person 1 har 6 dagar

TODO

---

4 3. Omlastningsmodell: probabilistiska bivillkor och heltal

4.0.1 3a) Nätverk och definitioner

4.0.2 3b) Generell modell

4.0.3 3c) Lösning med medelefterfrågan

4.0.4 3d) Probabilistiska bivillkor (90 %)

4.0.5 3e) Fasta kostnader för bruken

4.0.6 3f) Kapacitet på länkarna och heltal

---

5 4. Intermodal transportplanering

5.0.1 4a) Generell modell

5.0.2 4b) En färja (2000 lastbilar/år)

5.0.3 4c) Fler färjor

5.0.4 4d) Minskad kapacitet på vägen 2–3

---

6 5. Tomma godsvagnar

6.0.1 5a) Nätverk och modelltyp

6.0.2 5b) Generell modell

6.0.3 5c) Lösning

6.0.4 5d) Med och utan trängselvillkor

6.0.5 5e) Känslighet för P